AI SYSTEMS SCIENCE / AGENT SYSTEMS

32讲 · 零基础理论课程 • 第一部分：基本原理

04

记忆系统在智能体架构中的位置

Where Memory Fits in Agent Architectures

不只记住模型、工具与数据库的名字，而是理解每一个环节如何协调：读取过去、选择行动、观察世界、更新记忆，让一个长期智能体真正运转起来。

零基础入门 | 仅理论 · 不含实验 | 10张原理图 · 长周期案例 · 10题详解

COURSE ROADMAP

本讲学习地图

从完整系统认识六类功能组件，再沿控制循环理解记忆的读取与写入，最后学会诊断长周期任务中的失效原因。

00	本讲学习目标：画出一套会“动起来”的系统
01	1. 从一个真实会遇到的场景出发：科研助手忘了什么？
02	2. 先把整体模型看清：Agent不是一个LLM加一个数据库
03	3. 认识系统的两个世界：内部状态与外部环境
04	4. 六类组件怎样配合？先看一次完整的行动循环
05	5. 记忆与规划：一个负责过去，一个负责未来吗？
06	6. 记忆与推理：检索到的内容不会自动变成正确结论
07	7. 记忆与工具：工具是获取信息的接口，不是自动记忆
08	8. 记忆与环境：过去发生过不等于现在仍然如此
09	9. 记忆与控制器：真正重要的是流程和状态转换
10	10. 让这个科研智能体完整运行五轮
11	11. 三种典型架构：为什么没有唯一正确的模块拼法？
12	12. 一套能独立推导系统行为的简化数学模型
13	13. 记忆系统最容易被误解的10个地方
14	14. 一个故障诊断地图：最终失败在哪里发生？
15	15. 对照真实研究：这些论文分别关注架构的哪一部分？
16	16. 理论思考题：先自己推演，再看答案
17	17. 思考题详细答案与推理过程
18	18. 本讲总结：必须能独立重建的8句话
19	参考文献与延伸阅读（按本讲首次引用顺序）

贯穿本讲的动态主线

目标与当前状态 → 读记忆 → 提议行动 → 工具执行 → 环境反馈 → 更新状态与记忆

第04讲 记忆系统在智能体架构中的位置

课程：《智能体记忆系统：从零基础到科研理解》（32讲）
单元：第一部分 · 智能体记忆系统的基本原理
版本：v1.0 · 2026年10月17日
学习要求：只讲理论，不安排编程、实验、部署或使用软件；图中的智能体仅为教学抽象。
衔接：第01讲认识“为什么需要记忆与记忆循环”；第02讲理解“长上下文为何不能替代外部记忆”；第03讲区分“上下文、日志、知识库与记忆系统”；本讲更进一步：**记忆到底放在整个智能体系统的哪一环？它怎样和其他环节合作，又怎样把错误传递给后续行动？**

本讲学习目标：画出一套会“动起来”的系统

本讲最关键的学习目标，不是背诵几个模块名字，而是能在头脑中独立运行一个智能体：用户交给它长期任务以后，谁决定下一步做什么，谁执行动作，谁观察结果，谁保存重要信息，谁负责核对执行是否真的成功，哪些旧信息会在下次决策时重新出现。

学完以后，你应该能够独立解释：①“LLM”与“Agent系统”为什么不是同义词；②控制器、规划、推理、工具、环境和记忆分别承担什么职责；③一条历史记忆怎样进入下一步决策，某次工具执行结果怎样转化成新的记忆；④为什么系统已经保存正确记忆，却可能仍然采取错误行动；⑤为什么长期记忆不能代替环境实时状态；⑥发生系统故障后，怎样判断应该恢复执行进度、恢复历史知识还是重新核验外部世界。

> **本讲核心结论：记忆不是贴在模型旁边的“另一个大脑”，也不是孤立的向量数据库；它是一套跨时间维护信息、通过“读取”影响决策、通过“写入”吸收新观察的机制。真正让这些部件协作的是智能体的控制流程。**

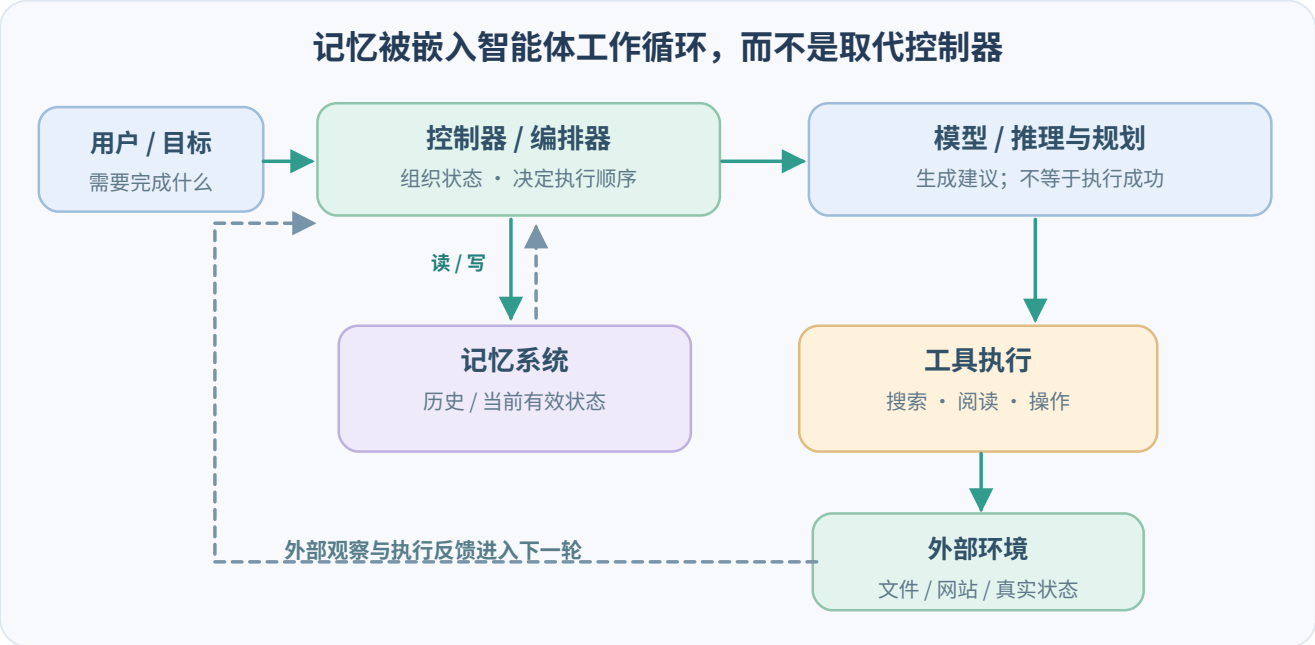


图1 · 教学抽象：控制器串联模型、工具和环境；记忆通过读取影响决策，通过写入保存新信息。箭头代表可能的数据流，不表示唯一实现。

1. 从一个真实会遇到的场景出发：科研助手忘了什么？

设想有一个**教学虚构**的长期科研助手。研究者提出任务：“本周帮我比较三篇智能体记忆论文，形成一份有证据的比较报告，暂时不要直接确定研究题目。”这一任务不是简单问答：它必须分批找论文、读方法、做笔记、核查作者的结果、比较各自局限、保留已完成的步骤，最后交付材料。

第一天，助手找到了论文甲并做了摘要；第二天阅读论文乙；第三天读取论文丙时，工具出现超时。第四天研究者继续询问：“现在进展如何？下一步该做什么？”

如果系统设计得不好，它可能出现以下情况：

- **错误A：重复执行**。因为不知道已经完成甲、乙两篇，重新检索和阅读全部论文。
- **错误B：凭空完成**。把“已启动读取丙”误记为“丙已精读完成”，最终报告缺失关键证据。
- **错误C：目标漂移**。忽略“暂不定题”的限制，自行开始给研究题目并以为这就是用户最初授权。
- **错误D：旧结论覆盖新结论**。第二天的临时比较被第三天发现的新证据推翻，系统仍保留旧版本。
- **错误E：遗漏环境改变**。即使记得曾下载过论文，却不知道现在文件损坏、路径已改变或者外部数据库中已出现新版本。

这五类现象不能一概说成“模型记忆力不足”。例如“重复执行”可能来自运行状态没有持久化；“凭空完成”可能是控制器把工具调用的发出视为成功；“遗漏环境改变”可能是没有重新检查外部状态。**当我们理解系统分工以后，才有能力进行合理归因。**

2. 先把整体模型看清：Agent不是一个LLM加一个数据库

大语言模型（Large Language Model，LLM）的基本工作是根据当前输入生成后续输出。这个输出可以是自然语言，也可以按约定格式表达“我建议调用哪个工具、参数是什么”。但“模型生成了工具调用请求”，不自动意味着现实世界里的工具已经执行，更不代表操作成功。

智能体系统（Agent System）则是把模型置于一个可反复运行的程序循环里：它接收任务、组织输入、决定是否调用模型、检查模型输出、选择和执行合法动作、接收环境反馈、更新状态，并判断是否继续或终止。

为了教学，可以把一个典型系统拆成六类组件。这里的“组件”指**功能职责**，并不要求工程实现必须是六个不同的软件包。

功能组件	通俗解释	核心职责	它不直接保证什么
模型 LLM	会提出文字与动作建议的推理引擎	对当前信息作语言理解与生成	不保证外部动作已经发生
控制器 Controller / Orchestrator	控制执行顺序的调度员	组织输入、调用模型与工具、维护状态、决定继续/停止	不自动理解所有论文内容
规划器 Planner / Planning	把目标拆成步骤的功能	生成、修订任务步骤与先后顺序	不保证计划本身正确可行
工具 Tool	可调用的外部能力	搜索、读取文件、查询数据库、执行程序等	不保证工具结果等于最终任务成功
环境 Environment	智能体需要影响或感知的外部世界	提供对象、状态和变化，例如真实文件、网页、任务系统	不按智能体的记忆自动变化
记忆系统 Memory System	跨时间的信息管理者	保存/筛选/检索/修订历史经验与任务信息	不保证被模型读到或正确使用

有些架构把工具和环境画成一个模块，有些把规划融入LLM的单次输出，有些把记忆作为控制器的若干服务，也有些让模型主动请求记忆操作。**只要职责与信息流可以解释清楚，这些架构都可能成立。**

2.1 用学校工作类比，但不要把类比当事实

把智能体想成一个承接科研任务的小型工作室：模型像给出分析意见的人，控制器像安排工作流程的协调者，规划功能像待办清单的制定过程，工具像检索系统和文档阅读器，环境像实际论文文件与研究进度管理平台，记忆像持续维护的工作档案。这样容易理解模块边界。

但这个类比也有限：模型不是一个拥有持续自主意识的人；控制器可能只是几十行或几百行程序；记忆并不一定会“自主”反思。重要的是**责任关系**，不是假定它们像人类同事一样具备心理能力。

2.2 谁真正执行动作？区分“建议”“批准执行”“成功”

假设模型生成：“调用文献读取工具，读取论文丙第3页。”接下来通常由控制器完成两件事：第一，检查工具名与参数是否合法、任务与权限是否允许；第二，向工具发出请求并等待实际结果。工具可能返回成功、超时、异常、无权限或者不完整内容。**只有回传的可核事实与任务验收条件相符，系统才有根据更新“读取成功”。**

把“模型想执行”当成“工具已执行”，或者把“工具返回某些文字”当成“真实任务已经完成”，是智能体系统常见的归因错误，和记忆容量无关。

3. 认识系统的两个世界：内部状态与外部环境

完整系统至少存在两类信息：**内部状态**与**外部状态**。内部状态是系统目前认为发生了什么、计划做什么，例如当前步骤编号、已完成清单、用户此前决定；外部状态是实际系统、文件、网站、工具或世界现在怎样。

例如，内部记忆写着“论文丙已经保存到文件夹”，而磁盘上文件已经被移动或下载中断。此时内部认知和外部现实发生了不一致。一个稳健的控制流程会依据任务风险决定是否重新读文件目录、校验文件，或检查原始数据，而不能假设“过去记得”就必然“现在仍为真”。

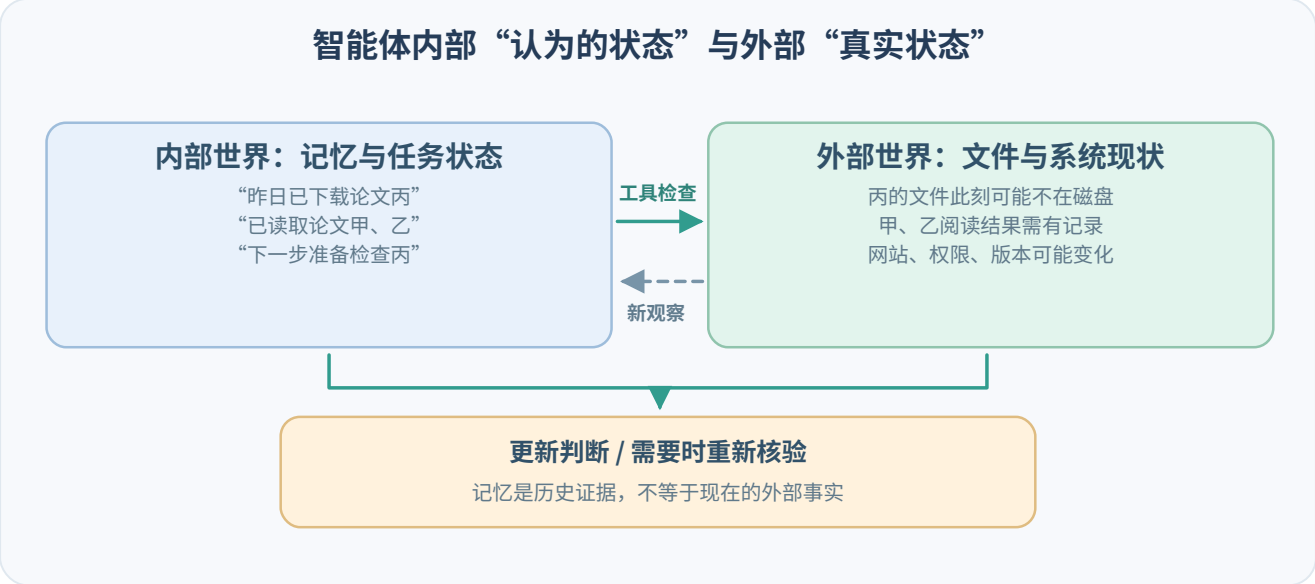


图2 · 存储的记忆与真实环境可在时间上失去同步，是否重新检查取决于变化概率、行动风险和所需证据。

3.1 智能体控制过程要解决的本质问题

控制器不断面对三个问题：**“现在处于哪个任务状态？”“下一步允许做什么？”“执行以后状态怎样变化？”**这三问可以分别联系到当前状态、行动选择与状态更新。

记忆对它的帮助主要是跨时间的信息：过去用户为何做出选择，哪些步骤曾成功，哪些方法失败，哪些约束持续有效。工具对它的帮助主要是取得外部新观察、执行外部动作。模型对它的帮助主要是解释输入和提出下一步行动建议。

3.2 内部状态不是只存自然语言

有些信息可写成自然语言：“用户不希望此刻确定选题。”另一些信息更适合结构化：`当前阶段=比较论文`、`已读取=甲/乙`、`丙状态=读取失败`、`下一步=检查丙的文件与工具错误`。后者并不一定要经过语义向量检索才能使用，可以由控制器直接读取明确字段。

因此“什么都放进向量数据库”并不是必然合理的记忆设计。 对当前执行进度这类状态，明确字段与可恢复的任务检查点往往更适合。未来课程会详细比较多种存储方式与其边界。

4. 六类组件怎样配合？先看一次完整的行动循环

下面不涉及程序代码，只把真实执行过程拆成连续事件。假设助手要回答：“丙论文的实验设计有没有支持作者关于记忆更新有效性的主张？”

步骤1：接收任务。 用户提出新问题；控制器读取当前任务标识和已记录的项目状态。

步骤2：准备模型可见输入。 控制器提取当前目标，从记忆系统读取有关丙论文的既往摘要、先前工具失败记录，以及用户要求“结论要有原文证据”的长期约束；再把相关内容组织为本次模型输入。

步骤3：决定下一步。 模型可提出“先确认PDF是否可读，再定位实验方法与消融部分”这样的建议。若此前记忆中的“已读取”与当前文件状态冲突，应优先核查。

步骤4：调用工具。 控制器检查工具调用是否合理，在允许的范围内调用文件读取或检索工具；不应把未发生的调用写成已完成。

步骤5：观察外部结果。 工具可能返回论文片段、页面范围、读取失败或路径不存在。结果是来自外部环境的**观察**，并不自带“判断正确”的保证。

步骤6：修订计划与记忆。 控制器与模型依据工具反馈更新任务进度：若已经完整读取所需方法部分，可以记录“完成指定段落阅读”；若只返回第2页，就不能保存为“全文已读”。重要事实写入时，还应保存来源与时间。

步骤7：继续、停止或求助。 控制器判断是否满足任务验收条件；如果缺少原文证据，可继续读取或明确报告未完成，而不是无根据地宣布成功。

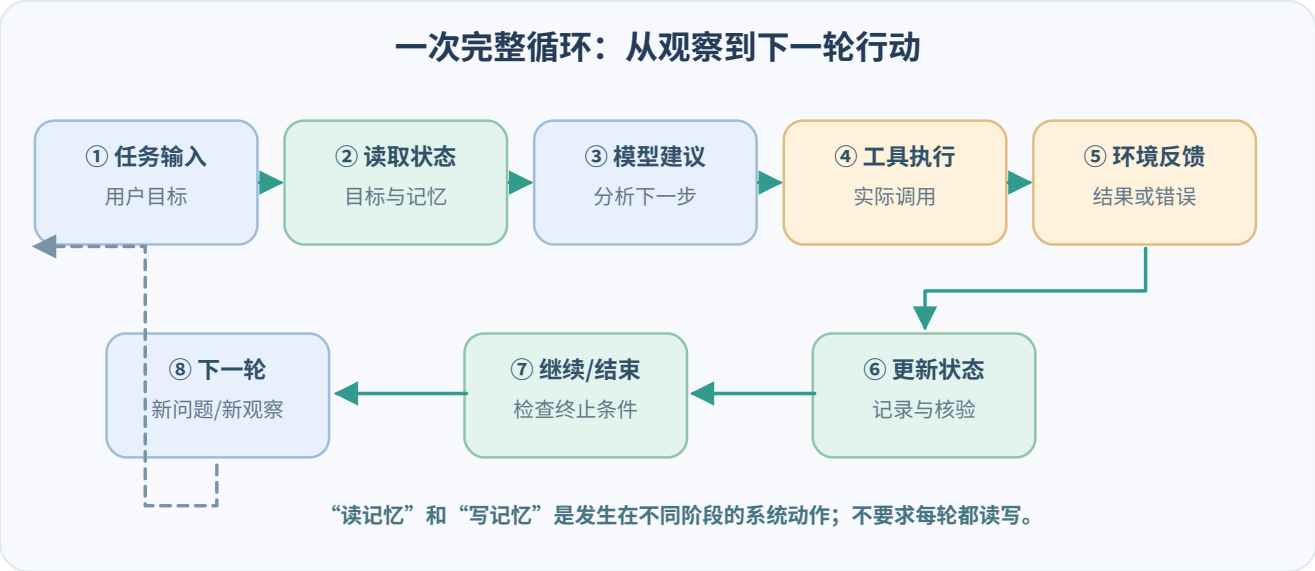


图3 · 一轮控制循环的八个教学步骤。箭头只是说明顺序关系；工程系统可合并或拆分步骤。

动态模型：用户/环境提供信息 → 控制器组织当前状态与记忆 → 模型提出下一步 → 控制器执行工具/动作 → 环境返回观察 → 更新任务状态与长期记忆 → 进入下一轮。**每轮前读、每轮后写并非强制规则**；是否触发由具体系统设计决定。

4.1 哪一步是“读取记忆”？哪一步是“写入记忆”？

上面第2步是在**读取**历史信息，第6步可能是在**写入或修订**历史信息。二者方向相反：读取把过去带到现在；写入把现在变成未来可用的过去。如果系统只能读不能写，它无法积累新经验；如果只能写不能取，已保存的信息对下一次决策可能没有用。

更进一步，即使能够读写，还需要判断**什么值得写、何时写、旧记忆是否失效、读取到的是不是最新版本**。这些问题构成后续课程关于记忆形成、检索、维护的主体。

4.2 “当前上下文”在哪里?

当前上下文不一定是一个独立数据库，它是在调用模型之前**被组装的本轮输入材料**。例如系统指令、用户问题、当前进度字段、检索到的经验、最近工具观察等都可以进入这一输入。LLM通常并非随时直接看见磁盘上全部材料，而只处理当前请求实际提供给它的内容。

5. 记忆与规划：一个负责过去，一个负责未来吗?

这个说法有帮助，但过于简单。**规划（Planning）**的主要任务是确定目标、子任务、顺序和约束；**记忆（Memory）**的主要任务是维护可供现在和未来使用的信息。规划经常要读取过去的经验，而计划本身也可以成为需要保存和更新的记忆。

举一个真实工作逻辑的教学例子：研究者要求“比较三种记忆架构”，规划器可以产生“搜集资料→阅读论文→比较机制→检查证据→写报告”的流程。记忆系统则记录“甲已阅读，乙的实验章节还没查，研究者要求排除缺乏原文证据的结论”。如果助手第二天继续，规划器不该重新安排已经完成的步骤，除非存在明确的复核要求。

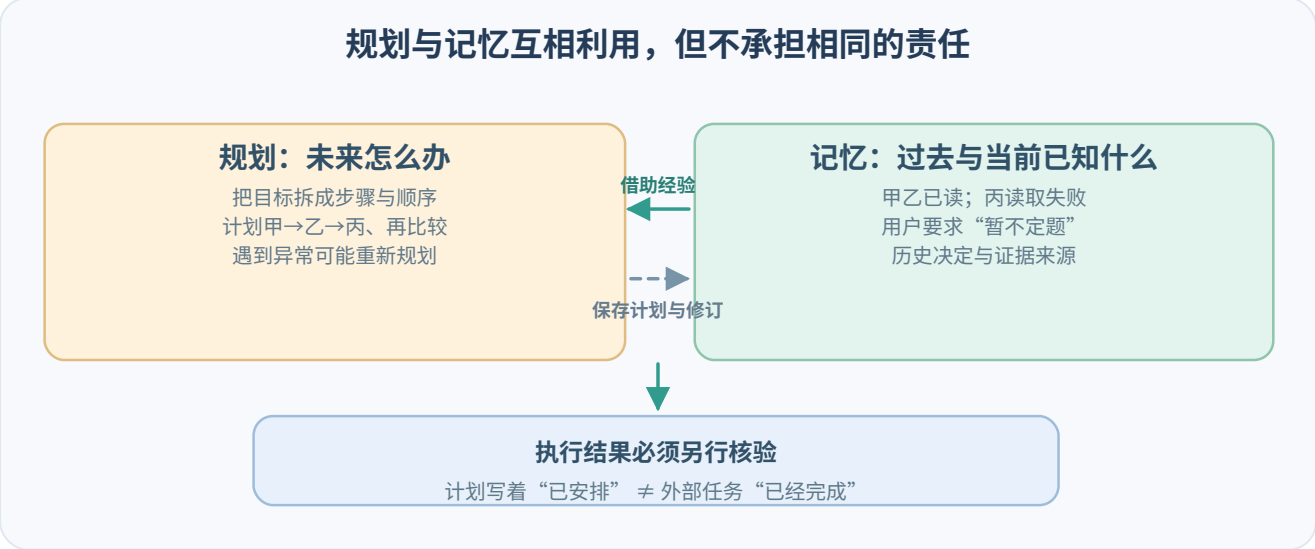


图4 · 规划与记忆是可交互的功能，不是“未来模块”和“过去模块”的绝对隔离。

5.1 为什么规划不是一次性写出清单？

现实任务会变化。假设读取论文丙失败，计划必须从“继续做报告”临时改为“先定位文件错误”；若研究者明确说“停止进一步下载”，计划必须立即遵守。这种根据新观察修订计划的过程可叫**重规划（Replanning）**。

记忆为重规划提供历史依据，但不是每次重规划都要调用专门的长期记忆检索。例如当前一轮的工具错误已经在上下文中，可能直接使用；若错误发生在几天前且本轮不可见，就需要读取保留下来的状态。

5.2 规划器必须是独立模型吗？

不必须。 在简单架构里，同一个LLM一次输出既有任务分析又有行动建议；复杂系统可能采用专门规划模型、规则、状态机、搜索算法，或者把任务拆成多个专门角色。区分模块是为了理解**功能**，不是规定实现必须有六个API或六个智能体。

5.3 计划也可能误导记忆

如果一个初步计划写着“第三天完成论文丙”，而实际第三天工具失败，系统不能因为计划表上出现“第三天完成”就将它作为事实记忆。**计划记录的是预期，执行日志记录的是发生，验收状态记录的是经核对的完成程度**；三者不能混写。

6. 记忆与推理：检索到的内容不会自动变成正确结论

推理（Reasoning）通俗地说，是根据当前材料分析问题、比较解释、得出判断。记忆提供的是可供分析的历史材料。智能体的正确结论至少依赖两方面：取得了合适的信息，以及正确利用了这些信息。

例如记忆检索返回两条记录：“第1天，研究者认为方案A可行”；“第5天，额外证据推翻了该假设，研究者暂停方案A”。如果模型仅凭第一条形成建议，是检索筛选或上下文组装问题；如果两条都看到了，却仍认为“A已批准”，则更接近信息理解、时间关系判断或决策推理问题。

6.1 区分三种经常混淆的句子

句子	它的性质	为什么重要
“论文作者报告方法A提高了指标X”	需要原文支持的文献陈述	作者实验的结论不是本团队复现实验结果
“我们推测方法A可能适合任务B”	基于已知材料的推测	推测不能写成已验证事实
“研究者决定暂时不采用方法A”	当前项目的决策信息	决策效力取决于主体、时间与最新修订

把这些句子全压缩成“方案A重要”，会丢掉来源、证据等级与决定关系。**记忆质量不仅在于是否留下一些关键词，还在于是否保留未来正确推理所需的关系。**

6.2 记忆能让模型变聪明吗?

外部记忆可以让本次推理获得过去不在上下文中的资料，但不自动改变底层模型参数，也不保证其逻辑能力提高。你可以把这理解为“考试时获得了过去的笔记”，不等于“推理方法本身变得正确”。

6.3 推理输出能否直接写回记忆?

可以作为候选材料，但需要分清是否已验证。模型推测“论文丙可能缺少消融”，可以标为“待核查推断”，不应直接写成“论文丙确实没有消融”。若系统每轮都把自己的猜测保存为事实，再用这些事实支持下一轮猜测，就有可能出现**自我强化的错误记忆**。

7. 记忆与工具：工具是获取信息的接口，不是自动记忆

工具（Tool）是智能体可调用的外部功能：文献检索、搜索网页、读取PDF、操作文件、运行程序、查询数据库、调用其他软件接口等。工具可以**读取或修改外部状态**。例如读取论文获取一段原文、创建一份分析报告、查询实验结果目录。

工具调用结果（Tool Result）通常是一条具体观察：“文件存在”“搜索到3条文献”“第4页包含某张表”。它可能成为记忆候选，但**工具结果本身不等于已经持久保存的长期记忆**。

7.1 从工具输出到可信记忆至少有三道区别

第一，**观察到什么**：读取器返回了哪些真实内容？第二，**这些内容意味着什么**：它是原文、摘要、错误信息还是模型解释？第三，**未来应当保存什么**：哪些结论已核实、哪些需要追溯、哪些只是临时噪声？

假设检索工具返回“某论文标题包含‘Memory’”。这不等于它研究了智能体长期记忆，也不等于它支持当前研究假设。还需要阅读真正相关的章节，并保存来源。把工具的表面结果直接写成完成状态，属于跨模块的语义错误。

7.2 为什么工具还涉及权限边界？

模型可以提出“删除旧文件”“访问云盘”等文字建议，控制器与工具层必须依据任务与授权限制实际能力。**记忆中出现某个历史授权描述，不代表此刻拥有执行新动作的权限。** 记忆是信息，权限应由实际系统机制与当前有效授权约束。

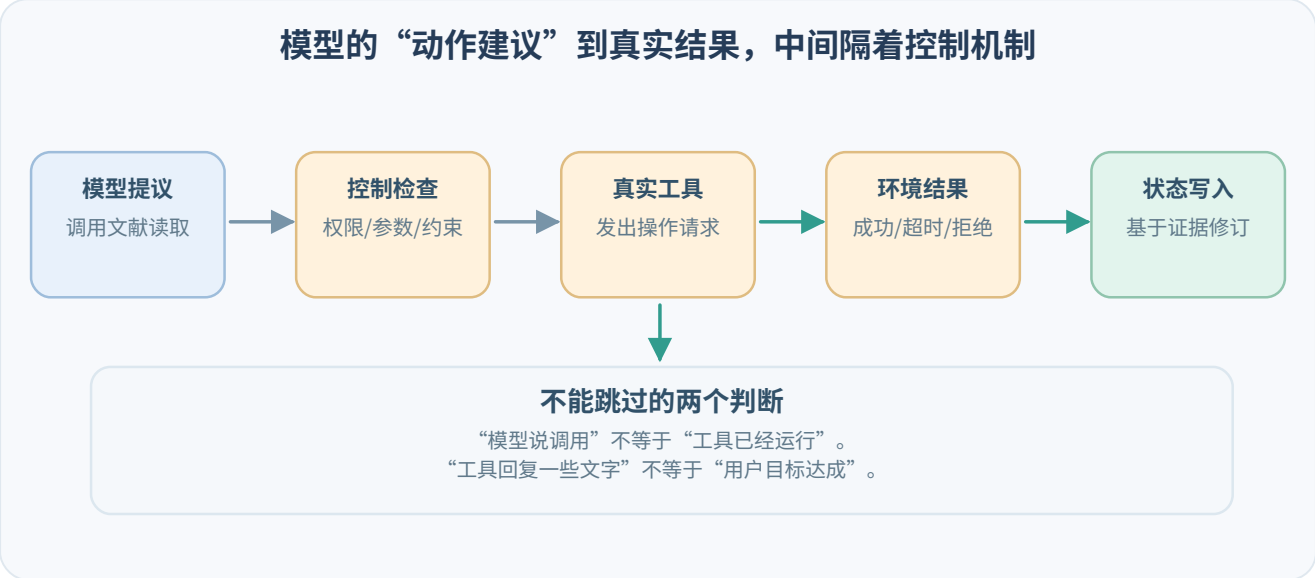


图5 · 记忆写入前要考虑工具结果、执行与验收的差别；控制器不应把提议误记为完成事实。

8. 记忆与环境：过去发生过不等于现在仍然如此

这里容易出现一个比数据库技术更根本的问题：**世界会变化，记忆只能描述曾观察到或推断出的状态。**环境（Environment）可理解为当前任务所接触的外部系统及其真实状态，例如论文网站上的最新文件、磁盘中现存的PDF、任务管理页面上的状态、实验结果所在的目录等。

例如第2天记忆保存“论文甲的PDF可读取”；第7天重新执行任务时，路径可能已变、权限可能已失效、文章可能更新。此时旧记忆最多说明“第2天曾可读取”，并不单独证明“第7天仍可读取”。

8.1 两条时间线：事件时间与当前检查时间

一条信息至少可能涉及：**事件发生时间（Event Time）**与**系统读取/写入时间（Processing Time）**。例如第2天发生的用户决定，在第4天才被记忆整理模块录入。这两个时间不同，会影响我们判断记录新旧。

如果一条旧决定晚于新决定被系统录入，仅按“写入时间最后”选择，可能错误地覆盖较新的决定。更可靠的做法需要考虑事件真实发生时间、来源、决定主体和是否发生显式撤销等因素。第31—32讲还将深入学习这一问题。

8.2 什么时候必须重新观察环境？

不是每一步都必须全面刷新。一个合理原则是：**外部变化概率越高、行动的不可逆性越大、记忆的信息越旧，越有必要实时核验。** 比如关于论文基本定义的知识通常无需每分钟重查；执行覆盖文件、提交收费请求或宣布实验完成时，不能只凭过去记忆。

8.3 “内部认为完成”与“外部实际完成”

假设模型生成了一份报告正文，但没有真正写入文件；任务状态却保存为 `报告已保存`。下一轮读取该记忆会延续错误。正确的跨模块关系应当是：工具返回文件创建结果，必要时读取文件检查，**再更新任务状态**。对有些任务，还需区分“文件存在”“内容完整”“内容准确”“用户已确认”不同验收层次。

9. 记忆与控制器：真正重要的是流程和状态转换

控制器（Controller，也常称Orchestrator，编排器）负责把若干步骤连接起来，可能包括：准备模型输入、调用模型、解析返回、执行合法工具、处理异常、更新任务状态、决定下一轮何时触发。没有控制器，模型即使能够说出“接下来检索历史”，也不意味着检索真的发生。

这里可以建立一个很有用的区分：**记忆系统管理“应该保留和复用的信息”；运行状态系统管理“执行到哪一步、当前是否可继续”**。二者既可能共用存储，也可能分开实现；职责区别比是否使用同一数据库更重要。

9.1 为什么“检查点（Checkpoint）”不能简单等于“长期记忆”？

检查点是某个工作流程运行到特定时刻的**状态快照**。例如助手完成甲、乙的阅读后保存“当前任务=比较论文；下一步=读取丙；已完成=甲乙”。若进程中断，检查点帮助系统恢复到接近中断前的位置。

长期记忆可能是“过去评估某种架构时发现的经验”“研究者持续采用的写作要求”“某研究项目的核心决定及证据”。它们可以跨任务、跨线程复用。**检查点偏向恢复一次执行，长期记忆偏向服务未来多次执行**，但具体框架的命名和边界可能不同。

以 LangGraph 的公开持久化文档为例，其文档明确区分使用**Checkpointer保存线程内图状态**与使用**Store存放可跨线程访问的数据**。这是一个真实工程设计实例，不代表所有Agent都必须用该框架。[5]

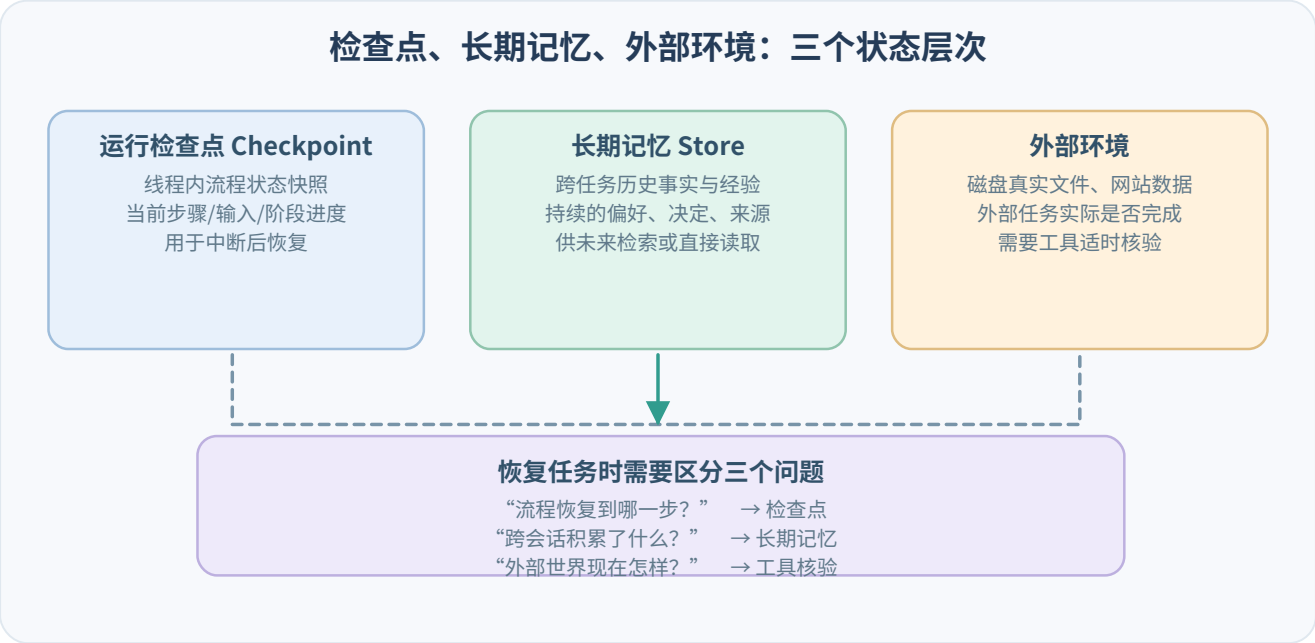


图6 · 以LangGraph的线程检查点与跨线程Store区分为启发的教学图；外部真实状态仍须独立核验。

9.2 宕机恢复后，为什么不能立即继续执行旧计划？

假设助手向外部系统发出“创建报告文件”的请求后瞬间断网，控制器没有收到成功回执。恢复时它可能看见检查点“文件创建中”。此时至少有两种可能：外部文件已创建但回执丢失；外部操作根本没有成功。**不能由“没收到成功消息”直接推断“必然没有执行”。**

稳健的恢复原则是：先查看可信外部状态，必要时使用可以识别重复请求的方式，避免把不确定操作随意重试。这里的关键不是模型“记不住”，而是分布式系统里动作与确认之间存在时间差与失败窗口。

9.3 为什么“不保存任何历史”和“全量保存日志”都不够？

只保存最终答案可能无法追溯错误；全量保留所有工具输出也会带来存储、检索与噪声问题。架构设计需要考虑哪些内容作为审计轨迹、哪些转成当前状态、哪些成为长期经验、哪些可安全清理。**原始证据与总结性记忆的不同用途**将贯穿后续各讲。

10. 让这个科研智能体完整运行五轮

现在把此前分散的知识组合起来。下面的日期、任务与成功状态仍然是**教学虚构**；目的是建立一个可在脑中逐步推演的流程，而不是描述某个真实AI的后台日志。

时刻	接收到什么/发生了什么	模型/规划的作用	控制器/工具的作用	记忆与任务状态如何变化
T0	研究者要求比较甲乙丙，暂不定题	拟出阅读顺序与证据标准	建立任务记录并初始化步骤	保存持续有效的目标和限制

时刻	接收到什么/发生了什么	模型/规划的作用	控制器/工具的作用	记忆与任务状态如何变化
T1	甲的论文资料可读取	识别需要看的方法与结果	调用读取工具，接收实际片段	记录甲哪些段落已查、对应来源
T2	乙论文阅读完成	比较甲乙的设计差异	保存阶段结果与检查点	当前进度为甲乙完成，丙未完成
T3	丙论文工具超时	解释错误，提议重新检查	记录超时而非假定成功	保存“丙读取失败，未验收”
T4	次日用户问“进展如何？”	基于现行状态生成答复	恢复检查点，读取必要记忆	回答“甲乙完成，丙失败，不能交完整报告”
T5	研究者要求先检查丙的文件	修订下一步计划	核验文件状态，获得实际反馈	更新任务状态；若事实变化，修订记忆

关注T3→T4：如果系统只保留了“原计划T3读取丙”，T4可能报错进度；如果系统保存了“丙工具超时”的真实结果并维持检查点，T4就能准确解释未完成原因。因此记忆有价值，**但价值来自整个闭环正确连接，不只是多了一份存储。**



图7 · 连续五轮的教学虚构事件。行右侧是可能的状态维护结果，不代表真实运行数据。

10.1 一条记忆的“旅程”到底有几站?

“丙的读取在T3失败”先由工具返回真实异常，控制器把它写为可追溯事件；任务状态标记 `丙=失败/未完成`；下次用户提问时控制器恢复该状态，把它放入模型可用的上下文；模型据此决定先核验，而不是直接写结论。这条信息经过了**观察→解释/归类→持久保存→重新读取→决策使用→可能修订**。

每个环节都可以失效：异常日志没采集，摘要漏掉“失败”，存储不可读，检索找到旧成功记录，模型误判，或工具执行没有核验。**同一个最终错误，内部原因可能完全不同。**

11. 三种典型架构：为什么没有唯一正确的模块拼法？

为了形成设计空间，我们把常见方案做成三种教学抽象。它们不是对所有论文的完整分类，也不是性能排行榜。

架构A：单轮问答模型。 用户输入→模型→输出。适合一次性的解释、改写、简单问题。主要依靠当前输入与模型参数，没有长期运行状态要求。

架构B：具备工具与控制循环的智能体。 用户任务→控制器→模型提议→工具行动→环境观察→下一轮模型。它能够完成多步任务，但如果没有可靠的持久化与跨任务信息复用，重启和长周期执行仍可能遇到问题。ReAct展示了交替进行推理和行动的一类方法；它不等于“任何ReAct系统天然具有高级长期记忆”。[2]

架构C：具有持久状态与动态记忆的智能体。 在B基础上维护任务进度、长期知识与经验；每次决策按需读取，行动之后按需更新。Generative Agents展示了在代理交互中结合记忆、反思和规划的一种研究路径；MemGPT则研究了在有限上下文中管理不同记忆层次。[3][4]



图8 · 三种典型设计的概念对照。是否需要长期记忆，应取决于任务要求与资源约束。

11.1 “多一个记忆模块”未必更好

复杂架构会产生额外的写入成本、检索成本、状态维护工作以及新的错误可能性。简单任务中，直接把充分可靠的信息给模型可能更好；长周期任务中，持久化与动态检索往往更有用。**选择架构，应该根据任务长度、信息变化速度、可靠性需求和资源约束，而不是仅凭模块数量多寡。**

11.2 正确比较时应该问什么？

例如“加入长期记忆后任务成功率更高”可能源自多方面：模型拿到了更多历史信息、任务重试方式改变、额外调用次数增加、控制器进行了校验、上下文预算增大等。科研判断需要进一步控制变量，才能区分“信息确实来自记忆设计”与“别的模块发生了变化”。这一原则先记住，实验评价将在后续课程系统学习。

12. 一套能独立推导系统行为的简化数学模型

零基础学习数学模型不需要马上掌握高级数学。下面先把符号解释成“程序里不同阶段的输入输出”，然后再看它们如何形成循环。

在第 t 次运行时，我们用 **G** 表示目标与约束，**O(t)** 表示当轮观察到的环境信息，**S(t)** 表示当前运行状态，**M(t)** 表示长期记忆，**C(t)** 表示交给模型的当前上下文，**A(t)** 表示系统选择并实际尝试执行的动作。

第一步：选出现在要看的材料。

```
C(t) = Assemble(G, S(t), O(t), Retrieve(M(t), question))
```

意思是：控制器把目标、当前状态、观察结果，以及根据当前问题从长期记忆中检索到的材料，组织为本轮可见输入。这里的`Assemble`不是某种固定算法，而是“组装输入”这类功能的名字。

第二步：选择候选下一步。

```
proposal(t) = LLM(C(t))
```

模型根据当前上下文生成建议。控制器还需要检查规则、权限和可用工具，才可能将建议变成真实动作。因此我们把“模型提议”与“外部执行”分开表示。

第三步：实际执行并观察结果。

```
O(t+1) = Observe(Execute(Validate(proposal(t))))
```

此式只是抽象地说明动作需通过检查、执行和观察；不是说所有工具必须经过完全相同的线性步骤，实际系统还会有异常、延迟、取消和并发。

第四步：维护系统状态与记忆。

```
S(t+1) = UpdateState(S(t), O(t+1))
M(t+1) = UpdateMemory(M(t), G, O(t+1), S(t+1))
```

注意，`UpdateMemory`并不意味着每一步都要写入，也不意味着模型权重发生变化；它只是描述外部记忆可能被修改的过程。有时仅更新运行状态而不更新长期记忆，反而更合理。

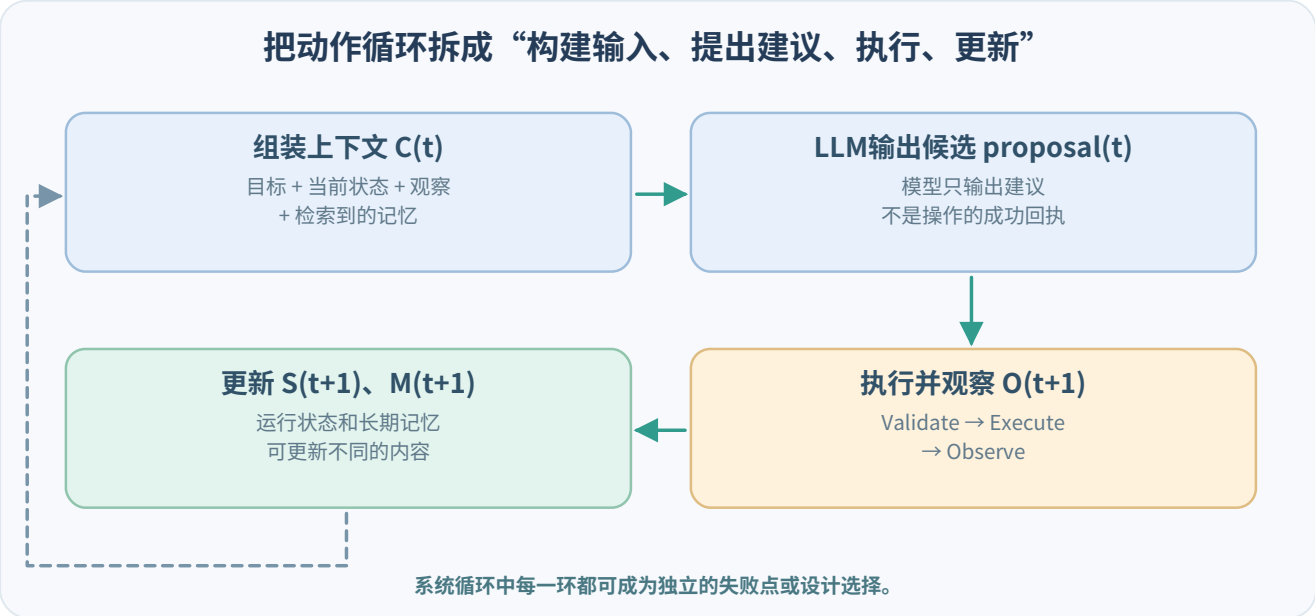


图9 · 第12节四组简化公式的直观图。它是理解信息流的框架，不是一套必须照抄的执行算法。

12.1 用模型预测一个错误

如果检索函数只返回第1天的旧计划，而忽视第3天的错误信息，`C(t)`里就缺少关键新状态；后续模型即使没有推理错误，也可能提出不合适的继续执行建议。另一个情况是`C(t)`包含完整状态，但模型仍误读：此时问题不在检索是否返回记录，而在模型的分析与控制器的后续检查。

这种“顺着箭头追踪信息”正是本课强调的**动态心智模型**。不必背公式，而要能根据一个环节的变化推导后续结果。

13. 记忆系统最容易被误解的10个地方

误区1：智能体就是大语言模型。 不正确。智能体通常是以模型为一个组件，由控制器、工具、状态等构成的系统。模型本身能生成建议，但执行与反馈需要其他机制。

误区2：有记忆就必然有规划能力。 不正确。能保存用户偏好，不代表能把复杂科研任务分解为可靠流程；规划能力也可以存在于缺少长期记忆的系统中。

误区3：规划会自动变成记忆。 不正确。计划必须被明确持久化才能跨进程留存；且计划是“预期”，不是“已完成”的证据。

误区4：工具返回成功文字就等于最终目标成功。 不正确。工具输出与最终任务验收要求可能不同，特别是在多步任务中。

误区5：检索到一段历史就已经完成“记忆使用”。 不正确。检索结果还需进入本轮可见输入并被正确解释，最终行动也可能失败。

误区6：记忆数据库里的信息永远是真实的现状。 不正确。记忆是历史信息或维护过的内部状态，外部环境可能变化。

误区7：所有信息都应写入长期记忆。 不正确。临时噪声、大量重复结果和未验证推断都可能增加后续干扰。还要考虑目的、成本、隐私与删除策略。

误区8：检查点就是一切长期记忆。 不正确。检查点主要保存某段执行流程的状态，跨任务共享的长期知识可能需要另一套组织方式。

误区9：智能体失败就是模型太弱。 不正确。工具、执行器、状态更新、检索、权限、环境变化和模型本身都可能引入错误。单看最终输出无法识别根因。

误区10：只要记忆系统足够复杂，智能体一定更强。 不正确。更多模块也会带来延迟、费用、新故障点与维护负担。要看任务是否真实受益。

14. 一个故障诊断地图：最终失败在哪里发生？

考虑最初的科研助手“报告里声称论文丙已阅读完成”，但实际上只读取到一页。至少可以列出以下不同解释：

层次	可能发生了什么	能帮助确认的证据
用户目标	原任务要求被误解为“只读摘要”	用户原话与目标解析记录
规划层	未安排检查全文、实验或补充材料	原计划及其后续修订
模型层	明知资料不全，仍判断已完成	实际提交给模型的输入与它的输出
控制器层	工具请求发出后就把状态改成完成	状态更新时序、回执和异常处理
工具层	读取器只返回部分页面或报错	工具原始调用、返回、异常信息
记忆写入层	把“已读取第1页”压缩成“已读全文”	写入前后内容、来源字段
记忆检索层	找回旧完成状态，遗漏后续纠正	检索候选、排序和实际命中
环境层	文件发生变化、路径不可用	文件状态、版本、大小等外部核验

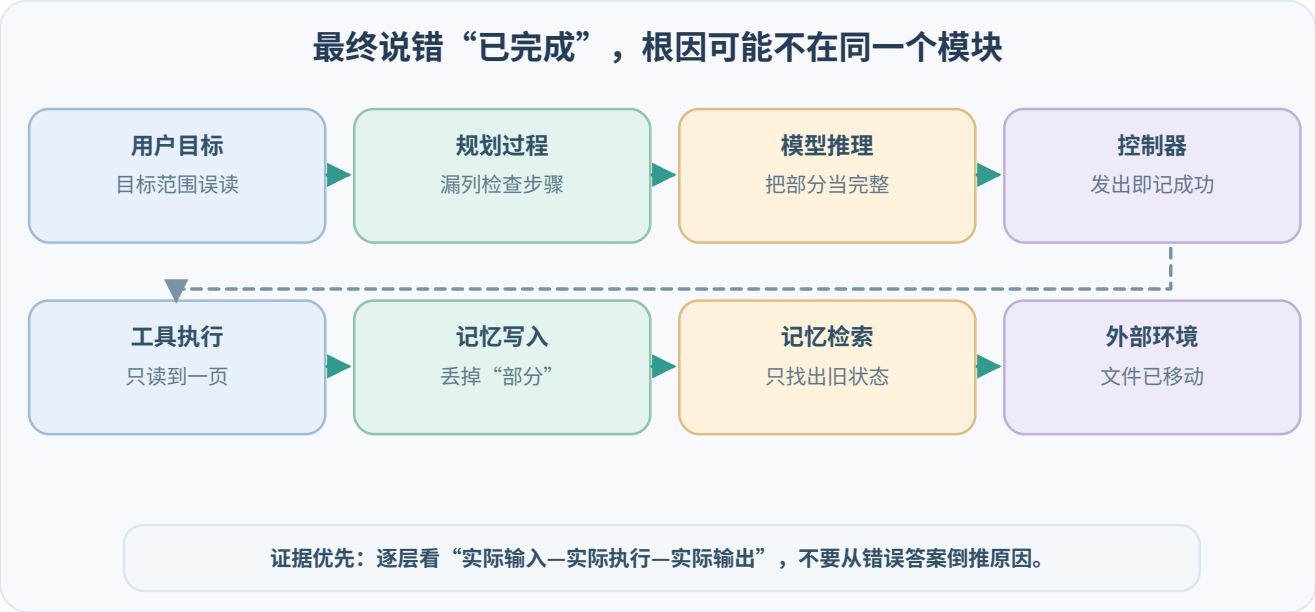


图10 · 错误可在任务解释、规划、推理、控制、工具、写入、检索和外部状态等不同层面出现。

14.1 为什么科研论文不能只报告“加入记忆后效果提高”？

假设一个系统不仅新增记忆检索，还更换了模型、加强了工具重试、扩充了上下文。这时任务成功率提高是真的，也仍然**不能单凭这些结果断言“提高一定来自记忆模块”**。真正研究机制时，要进一步定位信息流、比较条件、分析失败环节与替代解释。

我们现在暂不做实验，但先养成一个科研阅读习惯：看到“记忆有效”，就问**它究竟改善的是写入质量、检索命中、当前状态可见性，还是最终决策表现？** 不同命题需要不同证据。

15. 对照真实研究：这些论文分别关注架构的哪一部分？

以下不是让你在零基础阶段精读全部论文，而是帮助你知道“当前讲的架构不是随意画出来的”。以后读原始论文时，可以把新方法放回今天这张总图里。

- **CoALA（Cognitive Architectures for Language Agents）**：从认知架构角度分析语言智能体，将记忆、内部/外部行动与决策过程组织成可比较的概念框架。它适合作为今天这一讲的整体理论参照。[1]
- **ReAct**：研究如何在语言模型中交替进行推理与行动，以便在任务过程中获取新信息、调整后续行为。重点帮助理解“行动—观察—再决策”的控制循环，而非证明所有ReAct代理自带复杂长期记忆。[2]
- **Generative Agents**：通过存储经历、检索相关记忆、进行反思并参与规划，支持持续的代理行为。它强调记忆如何进入行为过程。[3]
- **MemGPT**：借鉴操作系统分层内存管理，将有限当前上下文与更大的外部记忆组织起来。它突出“上下文容量有限，怎样管理信息流动”的设计问题。[4]
- **LangGraph持久化文档**：这是工程文档，不是用来证明某个科研结论的同行评审论文。它具体展示了检查点与跨线程Store在职责上的区分。[5]

这些工作并不组成统一标准架构；它们各自关注的设计问题不同。**真正需要学会的，是看到某个方法就问：它管理什么状态？何时读取？何时写入？由谁决定？在哪个环节影响最终行动？**

16. 理论思考题：先自己推演，再看答案

题1｜模块定位。 研究智能体根据用户问题生成“搜索论文A”；程序真正向文献网站发出请求，并返回超时。请分别指出模型、控制器、工具、环境承担什么职责。为何不能标记“论文A已读取”？

题2｜计划与事实。 一个计划写着“星期三前完成三篇论文阅读”，但星期三只完成了两篇。如果这个计划被长期保存，星期四智能体应该如何理解它？

题3｜记忆与环境。 记忆显示“昨天本地有论文丙PDF”，今天工具发现文件不存在。应该信哪一个？昨天的记忆是不是必须删除？

题4｜故障归因。 系统保存着正确的“方案A已暂停”，检索也返回了它，但最终仍推荐立即执行方案A。最可能在哪些环节进一步查证？

题5｜检查点与长期记忆。 一个科研助手执行到第4步时进程退出。重启后想知道“上一步做到哪里”和“三个月前研究者拒绝过哪种方法”，这两个问题使用的是不是同一类记忆职责？

题6｜工具与记忆。 PDF解析器返回一段论文摘要，智能体立即写入“论文证明架构B优于所有方法”。至少存在哪两层问题？

题7｜最小架构选择。 仅需回答“Embedding是什么”，是否一定要引入可自我更新的长期记忆系统？为什么？如果任务改为三个月连续跟踪一个研究项目呢？

题8｜自主恢复。 智能体发出文件写入请求以后网络断开，既无成功也无失败回执。它可否直接重复写入？应该先检查什么？

题9｜公式推导。 假设 `Retrieve(M(t), question)` 遗漏了最新停止指令，而当前状态 `S(t)` 也没有保存停止标志。推演模型为什么可能执行已被撤销的操作；再分析哪种额外的控制器机制可以降低风险。

题10｜科研辨析。 论文声称新记忆架构改善成功率，但新架构同时增加了一倍工具调用与多次人工检查。为什么不能立即把效果归因于记忆结构？

17. 思考题详细答案与推理过程

答1。 模型生成的是搜索建议；控制器解析建议、检查调用条件并发出请求；工具是实际访问文献网站的接口；环境包括文献网站及其当前状态。工具超时代表缺乏可核实的阅读结果，不能仅因计划执行过一次就写“已读取”。可记录的是“搜索/读取尝试发生且超时”，并保留错误信息供下一步决策。

答2。 星期三完成三篇是过去制定的**计划或预期**，不是已发生事实。星期四应结合执行日志/检查点识别“实际完成两篇，第三篇待处理”，按需要调整计划。计划可以保留，以解释原本目标和延期原因，但不能覆盖实际验收进度。

答3。 对“今天文件是否存在”这个实时问题，优先采信今天可信工具的实际检查。昨天“曾经存在”的历史仍可能是真实的，不应简单删除；可以维护新的当前状态“今天读取时文件不存在”，并保留时间信息。历史记录与现行状态是不同命题。

答4。 首先查看这条记忆是否真的进入**本轮模型可见上下文**；再看上下文是否同时存在相互冲突的旧指令；然后检查模型如何解释以及控制器是否阻止未经授权的旧动作。仅证明检索返回正确文本还不足以证明后续所有环节正常。

答5。 “执行到第4步”主要需要当前任务的运行检查点和执行状态；“三个月前拒绝某方法的原因”需要跨任务或跨会话保存的历史决定、证据及可能的长期记忆检索。它们可以保存在同一个数据库，但**信息的目的、访问范围与更新方式不同**。

答6。 第一，工具仅返回摘要，不能自动证明原文实验设计与数字可信；第二，“架构B优于所有方法”是过度泛化的推断，可能超出原文对比范围。即使引用无误，也要保留比较条件、指标、任务、基线以及事实与推断的区分，不能直接写成普遍事实。

答7。 单轮概念问答通常可以直接由模型根据当前输入回答，长期可自编辑记忆未必必要且增加复杂度。三个月连续项目则涉及用户决定、进度、证据版本、失败教训和跨会话恢复；持久状态与可检索历史的价值提高。不过仍需按任务区分运行检查点和长期记忆，不能因为长周期就默认最复杂方案最优。

答8。 不能未经检查就认为原写入必然失败。可能出现“服务器已成功写入，只是客户端没收到回执”。应先查询实际文件状态、标识与内容，结合幂等操作或重复检测策略决定是否再次执行。单纯保存“未收到成功”不足以判定世界状态。

答9。 新停止指令既没有在任务状态里标记，也未被记忆检索返回，因此组装的上下文缺少它，模型可能继续依旧计划提议动作。可降低风险的机制包括：把当前有效停止/授权状态保存在控制器直接读取的权威状态中，并在工具调用前强制检查，必要时拒绝执行。这说明**权限与停止条件不应只依靠语义检索的偶然命中**。

答10。 因为成功率变化可能同时受到额外工具调用、人工检查、模型输入变化、预算增大等因素影响。要解释记忆结构本身的机制，需分辨不同条件造成的影响，并确保比较的任务、可见信息、资源和验收口径具有可比性。本讲只要求提出正确的归因问题，后续课程会讲评估理论。

18. 本讲总结：必须能独立重建的8句话

1. **模型是一部分，智能体是一个持续执行与反馈的系统**；模型的输出不等于外部动作成功。
2. **控制器维系流程**：准备输入、调用模型、执行工具、处理反馈、维护任务状态与终止条件。
3. **规划关心下一步要做什么**，但计划只是预期；实际完成以执行反馈与验收状态为准。

4. **记忆把过去的信息带进现在的决策**，把新观察保留给未来；读取和写入是不同操作。
5. **工具负责感知或影响外部世界**，返回的数据要判断是否完整、真实、相关，并不自动成为长期记忆。
6. **环境会变化**：旧记忆可以作为历史证据，但不能代替高风险行动前对当前外部状态的检查。
7. **检查点帮助恢复当前执行，长期记忆帮助复用跨时间知识**；二者有交叉，但职责不同。
8. **最终行为错误可能来自任何一层**。研究智能体记忆时，必须顺着数据与控制流识别真正的机制。

> **贯穿课程的心智模型：外部世界产生观察 → 控制器组织目标与当前状态 → 按需读取记忆 → 模型分析并提出下一步 → 控制器执行与核验 → 更新进度及必要的长期记忆 → 下一轮继续。** 先能在脑中让它运行，再学习具体数据库和检索算法。

下一讲预告：第05讲进入第二部分《理解记忆系统必需的基础技术》，从Token、上下文窗口和信息容量展开，为以后学习记忆编码与检索机制打基础。课程不因开始技术基础部分就立即安排编程或实验。

参考文献与延伸阅读（按本讲首次引用顺序）

[1] Sumers, T. R., Yao, S., Narasimhan, K., & Griffiths, T. L. Cognitive Architectures for Language Agents. arXiv:2309.02427，2023（CoALA概念框架；论文版本、最终发表信息以原文为准）。https://arxiv.org/abs/2309.02427

[2] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. https://arxiv.org/abs/2210.03629

[3] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. Generative Agents: Interactive Simulacra of Human Behavior. UIST 2023. https://doi.org/10.1145/3586183.3606763

[4] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023. https://arxiv.org/abs/2310.08560

[5] LangChain. LangGraph Persistence Documentation. 持久化、Checkpointers与Stores官方工程文档（2026-10-17访问）。https://docs.langchain.com/oss/python/langgraph/persistence

阅读边界说明：本文的“科研助手”情节、日期、模型公式与图示为帮助理解的**教学抽象**，并非原论文的逐字架构图、实验数据或真实运行轨迹；参考文献用于支撑所涉及的代表性设计思想与工程概念，不构成对某个具体方案性能的验证。